Response to Reviewer 1

Enhancing Consent Policy Privacy in Blockchain-Based Healthcare Data Sharing

---

Dear Reviewer 1,

We thank you for your careful review and constructive comments, which have helped us to improve the manuscript considerably. Our point-by-point responses follow and a marked manuscript in which all new or revised text appears in red is provided.

Comments 1: *The security implications of the unbound program length should be more strongly addressed. According to the manuscript, the preimage of the commitment does not include the length of the program ℓ. Thus, a TPS that deviates from the prescribed workflow may consider a stack-consistent prefix of the committed token array instead of the full desired policy. This is not an implementation detail, but a matter of soundness. The proposed framework is oriented toward verifiable consent enforcement and therefore the authors should either bind the program length in the commitment/proof relation in the current implementation or give a much stronger justification for postponing this modification.*

Response 1: We agree and have bound the program length in the current implementation. The commitment is now $h = \mathsf{Poseidon}(\mathsf{H}_{\mathrm{pol}}(P_{\mathrm{enc}}) \parallel \ell \parallel r)$, and the circuit supplies the same private ℓ to both the commitment check and the interpreter, so an accepting proof can no longer evaluate a shorter stack-consistent prefix of the committed program. The change adds 88 constraints (about 0.12%) and no public input. The statement that this binding was left to future work has been removed.

Revised in: Section 5.3, first paragraph (pages 13–14, lines 503–516); Section 7, first paragraph under "Policy and context binding" (pages 33–34, lines 1190–1194).

Comments 2: *The proof must be cryptographically linked to the authorization context. The current proof statement is not linked to the requester identity, protected resource, or session context. As mentioned in the manuscript, single-use of the nonce does not allow reuse of the same session parameters, but it does not prevent replay of a valid proof captured by an active party using fresh parameters. This is a significant limitation of the authorization guarantees of the current prototype. The authors would ideally add the relevant requester/resource/session values to the proved statement and experimentally evaluate the resulting circuit/public-input and gas cost changes. If this cannot be implemented in the current version, the security claims and deployment implications should be made even more explicit throughout the manuscript, including the abstract and conclusion.*

Response 2: We have implemented the binding in the circuit the contract. The proof now carries a public context digest $d_\chi = \mathsf{Poseidon}(id_{\mathrm{req}} \parallel id_{\mathrm{res}} \parallel \nu)$, which the contract recomputes from the transaction parameters before verification. A captured proof submitted with a fresh nonce, another resource, or another requester is therefore rejected within one deployment; our end-to-end tests confirm this. As requested, we measured the cost: 605 additional constraints and one additional public input. At Amoy's gas schedule, the binding adds 64,022 gas per access, and recording the bound context on-chain adds 68,946 gas, together 18% of the mean unpadded access cost. The padded cost is unchanged because the unpadded access cost remains below the padding band. The Discussion states the remaining scope: the digest does not yet cover the submitter or a chain/contract domain.

Revised in: Section 4.4, first requirement (pages 10–11, lines 377–395); Section 5.3, second and third paragraphs (pages 14–15, lines 517–553); Section 4.2, last paragraph (page 10, lines 353–357); Section 7, second and third paragraphs under "Policy and context binding" (page 34, lines 1196–1204); Table 15 (page 31).

Comments 3: *The problem of attribute-origin requires further investigation. The issuer-signature mechanism is a placeholder for consistency, based on Poseidon, and thus does not provide cryptographic unforgeability as it stands. Thus, the proof proves the consistency of the given witness but does not prove that the private attributes are derived from an authorized issuer. This difference is particularly important in a healthcare authorization system. The authors should clarify which authorization properties are guaranteed by the prototype they implemented, and which are dependent on external trust. Even an analytical assessment of a concrete design for the integration of real credential verification would greatly improve the work.*

Response 3: We replaced the placeholder with an in-circuit EdDSA-Poseidon credential check. The circuit verifies the issuer's signature over the policy commitment, requester pseudonym, resource, claims, validity interval, and credential identifier; it checks the issuer key against a commitment bound to the policy and derives the profile-backed DUO attributes from the signed claims. A compromised TPS can therefore no longer alter these attributes; all 25 targeted tests pass, including forged, expired, and substituted credentials. We also state what remains trusted: the prototype does not prove that the issuer key belongs to an accredited healthcare authority, and it does not check revocation. The Discussion now gives a concrete integration design with a governed issuer registry, a holder-signed request context, and a private non-revocation proof.

Revised in: Section 4.1, fifth paragraph (page 8, lines 306–318); Section 5.4 (pages 15–16, lines 555–575); Section 6.3, paragraph "Shared corpus" (page 20, lines 767–769); Section 7, second paragraph under "TPS trust and credential provenance" (page 33, lines 1155–1167) and paragraphs under "Issuer governance and deployment path" (page 33, lines 1168–1189).

Comments 4: *The trusted proof service is still an assumption of trust. The privacy contribution should be viewed in the context that the TPS has access to confidentiality-sensitive information and builds the witness. The implementation also co-locates the TPS with the provider, concentrating this trust. Thus, a compromised or colluding TPS may replace attributes or refuse to compute a proof. This problem is acknowledged in the discussion, but the practical implications require a more comprehensive discussion. The authors should clarify the difference between the cryptographic guarantees provided by TPS-ZKPI and the guarantees based on honest TPS behavior. A small threat/guarantee matrix would help clarify things.*

Response 4: We have made the distinction explicit and added the requested matrix. Section 4.1 now separates authorization correctness, which the commitment and on-chain verification enforce regardless of the TPS, from policy confidentiality and liveness, which depend on the TPS following its workflow. The new Table 3 states, for each actor or failure mode, whether each guarantee is cryptographic, TPS-dependent, external, or not provided. The rows cover the public observer, the requester, an honest TPS, a compromised or colluding TPS, forged credentials, and unauthorized or revoked issuers. In short, a compromised TPS cannot break authorization correctness but can breach confidentiality and refuse service; the Discussion addresses the resulting accountability gap.

Revised in: Section 4.1, third and fourth paragraphs (page 8, lines 283–305); Table 3 (page 9); Section 7, paragraphs "TPS trust and credential provenance" (page 33, lines 1148–1154) and "Future directions" (page 34, lines 1205–1212).

Comments 5: *Empirically-grounded claims of privacy must be carefully scoped. The evaluation assumes a passive, metadata-only public-chain adversary, and the effective policy-dependent attack features ultimately reduce primarily to commit and access gas measurements, as several other transcript characteristics are constant by construction. The authors rightly point out that their threat model does not cover timing of transactions, ordering of transactions, mempool/broadcast behavior of TPSs, off-chain communication of the TPS, adaptive probing of requesters, and compromised behavior of TPSs. These exclusions are important. The wording should not be interpreted as establishing general policy confidentiality, and the result should be qualified as empirical resistance against the modeled public-chain metadata attacks.*

Response 5: We agree and have adopted this formulation. The Abstract, Discussion, and Conclusion now present the result as empirical resistance against the modeled public-chain metadata attacks. The Conclusion lists every excluded channel and states that the results do not establish policy confidentiality against them. The evaluation was also re-run on the revised system (1,773 Polygon Amoy executions). Of the 45 supervised tasks, 43 pass; we report the two failures and trace them to chance correlations between the random padding targets and the labels (subtracting the logged target returns both to PASS).

Revised in: Abstract (page 1, lines 2–18); Section 7, paragraphs under "Privacy findings" (pages 31–32, lines 1100–1122); Section 8, third and fourth paragraphs (pages 34–35, lines 1229–1246).

Comments 6: *Provide additional justification for the PASS/FAIL leakage criteria. In the manuscript, a failure is defined only if the mean AUC is greater than 0.55 and at least 50% of the 30 seeded permutation tests are significant. This leads to a criterion based on reproducibility, although the exact thresholds seem to be application-defined. The authors need to better justify the choice of 0.55 and 50% and ideally perform a sensitivity analysis to show that the overall conclusions are robust to reasonable alternative thresholds.*

Response 6: The methodology now justifies both thresholds. The AUC threshold of 0.55 is an effect-size floor that keeps a significant but negligible AUC excess from triggering a FAIL. The 50% threshold requires significance in at least 15 of the 30 policy-disjoint splits, far above the 5% rejection rate expected per split under the null, so a FAIL reflects an effect that recurs across splits. In the evaluated run, the stability condition is the binding one: the AUC condition alone would flag three additional PCH and three additional SPP tasks, each significant in fewer than half of the splits. A new sensitivity analysis re-applies the criterion over AUC thresholds of 0.52–0.60 and significant-seed fractions of 0.3–0.7. At fractions of at least 0.5, at most the same two tasks fail; lower fractions add at most two SPP contrasts, and the PI verdict never changes. A null calibration with policy-independent gas features gives a spurious-failure rate of about 3% per PCH task and none for SPP. The overall conclusions are therefore robust to reasonable alternative thresholds.

Revised in: Section 6.2.1, paragraph "Operational thresholds and sensitivity" and the two paragraphs that follow (page 19, lines 698–725).

Comments 7: *Can you expand on the real cost of randomizing gas padding? The gas padding ablation is important, as it shows that policy-dependent gas signals are more recoverable when the padding is removed. But it costs about 38% in access-gas and 40% in commit-gas overhead to provide this protection. The manuscript would benefit from a more explicit discussion of this privacy-cost trade-off, e.g. whether the overhead remains practical for realistic transaction volumes and whether the padding strategy remains effective when policies or circuits become more complex.*

Response 7: We have expanded the cost analysis with new measurements. On the rerun ablation, padding raises mean access gas by 46.1% and mean commit gas by 38.4%. For transaction volume, the Discussion gives per-request figures from which any volume follows: a padded access costs about 0.032 POL at 30 gwei, so the on-chain cost grows linearly with the number of requests, and a measured throughput of 0.936 proofs/s lets one prover handle about 80,900 requests per day on a single host. For complexity, circuit sweeps (program length 32–128, ontology depth 4–16) and four policy classes using up to all 11 predicate slots keep the median proving time below 0.8 s. The public interface stays the same in every configuration, so the padding strategy applies unchanged.

Revised in: Section 6.9, paragraph "Cost" and the three paragraphs that follow (pages 30–31, lines 1066–1096); Tables 16 and 17 (page 31); Section 7, paragraphs under "The cost of padding" (pages 32–33, lines 1123–1147).